Navigating Complexity with GenAI

Radu Dumbrăveanu

Bălți · BitConf 2026



MOTTO

There has never been a more interesting time to think than in the age of AI.

Niciodată nu a fost mai interesant să gândești decât în epoca inteligenței artificiale.

Prin complexitate mă refer la:

- sisteme / fenomene / probleme cu mulți factori interconectați
- granițe difuze între factori (greu de delimitat)
- combinații care nu pot fi prevăzute exhaustiv
- și situații care doar par complexe: sunt noi pentru mine sau formulate neclar

Cum abordăm complexitatea?

- **A.** înțelegem fenomenul din spatele problemei și folosim proprietățile lui, unde putem
- **B.** ce rămâne, delegăm altcuiva:
  - **AI**

A. Cum rezolvăm o problemă (complexă)? (Pólya)

Cele 4 etape

1. înțelege problema
2. elaborează un plan
3. execută planul (verifică fiecare pas)
4. privește înapoi
    - verificare rezultat, reutilizare la alte probleme

Euristici

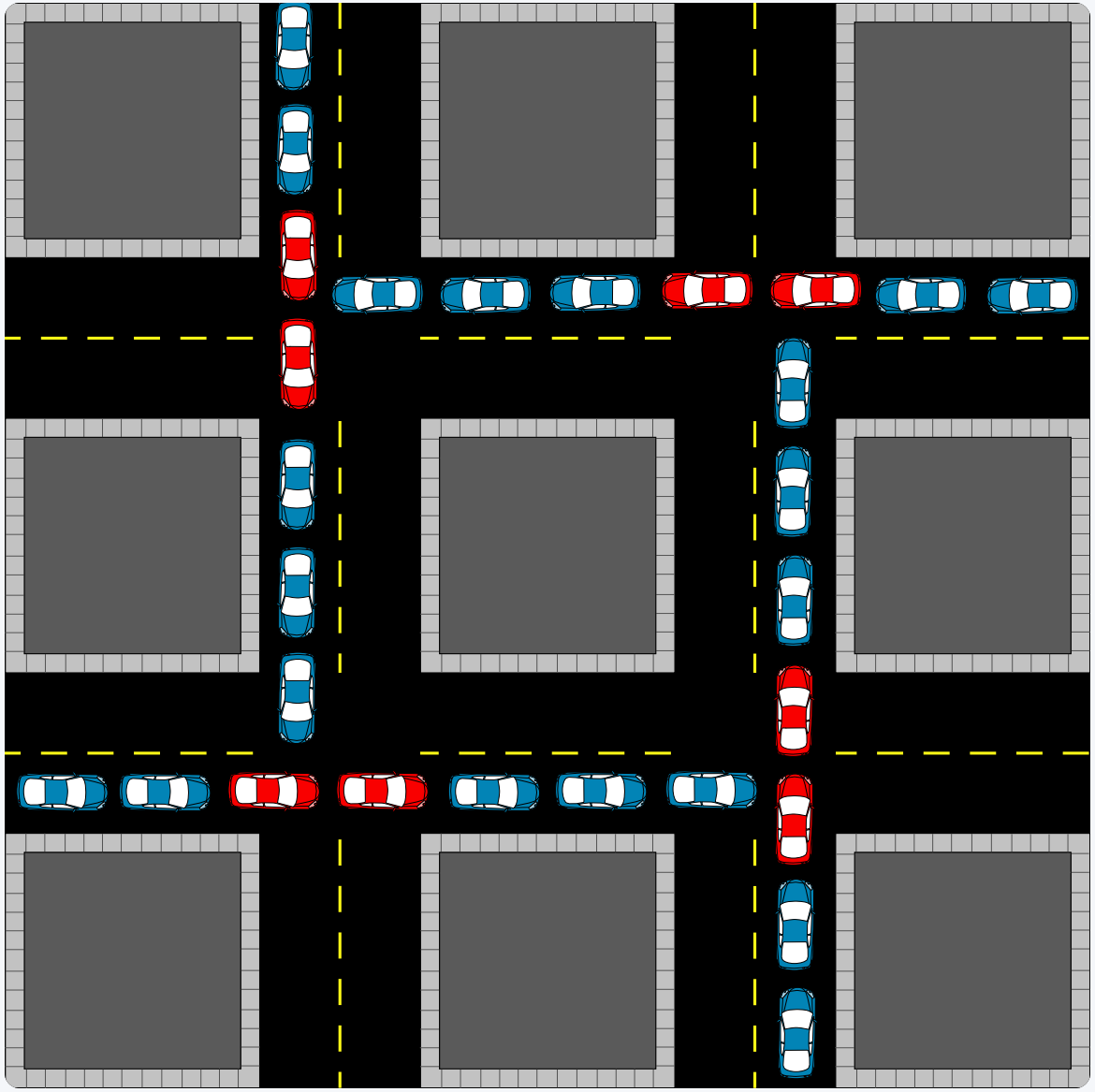
- problemă asemănătoare / analogie
    - poți folosi o problemă înrudită deja rezolvată?
- **specializare** (cazul particular)
- generalizare
- **problemă auxiliară** (mai simplă)
- descompune în părți
- desenează o figură

George Pólya, *How to Solve It* (1945). Lista de euristici nu e exhaustivă: dicționarul lui Pólya are 67 de intrări.

Exemplu: deadlock (interblocare)

- din experiența mea: eroarea „nu mai sunt conexiuni pentru baza de date”
- mai multe procese gestionează resurse, printre care și conexiunile la baza de date, luate dintr-un pool de N conexiuni (număr limitat)
- procesul A ține o conexiune și mai cere o resursă
- resursa e ocupată de procesul B
- B, înainte să o elibereze, are nevoie de o conexiune la baza de date
- fiecare așteaptă după altul: nimeni nu mai înaintează

Sursa desenului: https://commons.wikimedia.org/wiki/File:Gridlock.svg



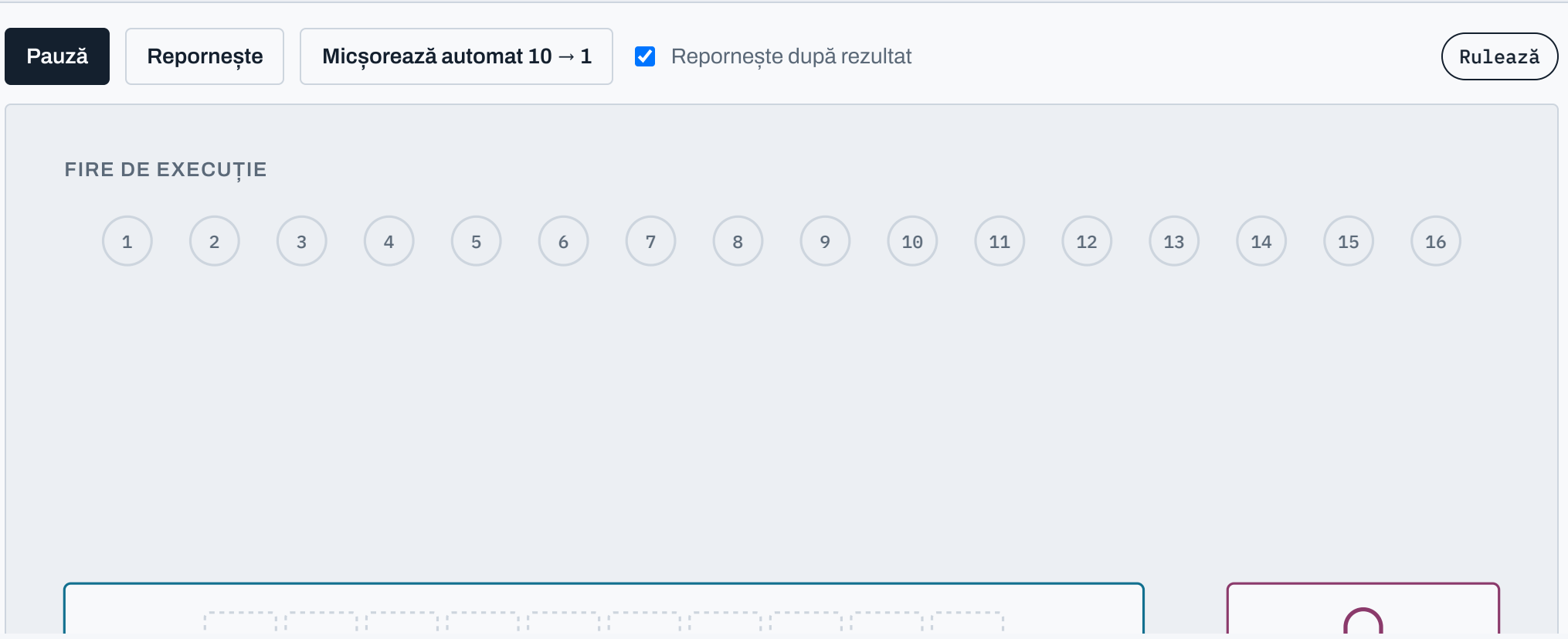
Deadlock: de ce e greu de studiat?

- mulți utilizatori: sistem greu de depanat
- puțini utilizatori: ușor de depanat, dar greu de reprodus

Simulare generată de Claude: pool de conexiuni, micșorat de la 10 la 1 · deschide direct

Deadlock în pool-ul de conexiuni

Un fir de execuție ține o conexiune la baza de date și are nevoie de o resursă ținută de alt fir. Acel fir eliberează resursa doar dacă primește o conexiune liberă. Micșorează pool-ul și urmărește cum se apropie de 100% șansa ca blocajul să apară.



Deadlock N=1 vs. AI

- la N=1 fenomenul **converge singur spre starea dorită**: problema se reproduce și e ușor de studiat
- alt exemplu: OOM
  - uneori sistemul afișează mesajul `OutOfMemory`
  - soluție: micșorăm memoria totală și eroarea se reproduce mult mai repede
  - astfel se vede mai clar cauza: cine cere multă memorie sau cine ocupă prea multă memorie

Deadlock: cazul particular în locul celui general (specializare)

- nu am delegat problema către AI
- în loc de problema generală, am luat un caz particular, mai simplu (N=1, memorie mică)
- cazul particular **converge singur spre starea dorită**, deci s-a dovedit mai ușor de reprodus și de studiat
- prin el am găsit soluția și pentru problema generală

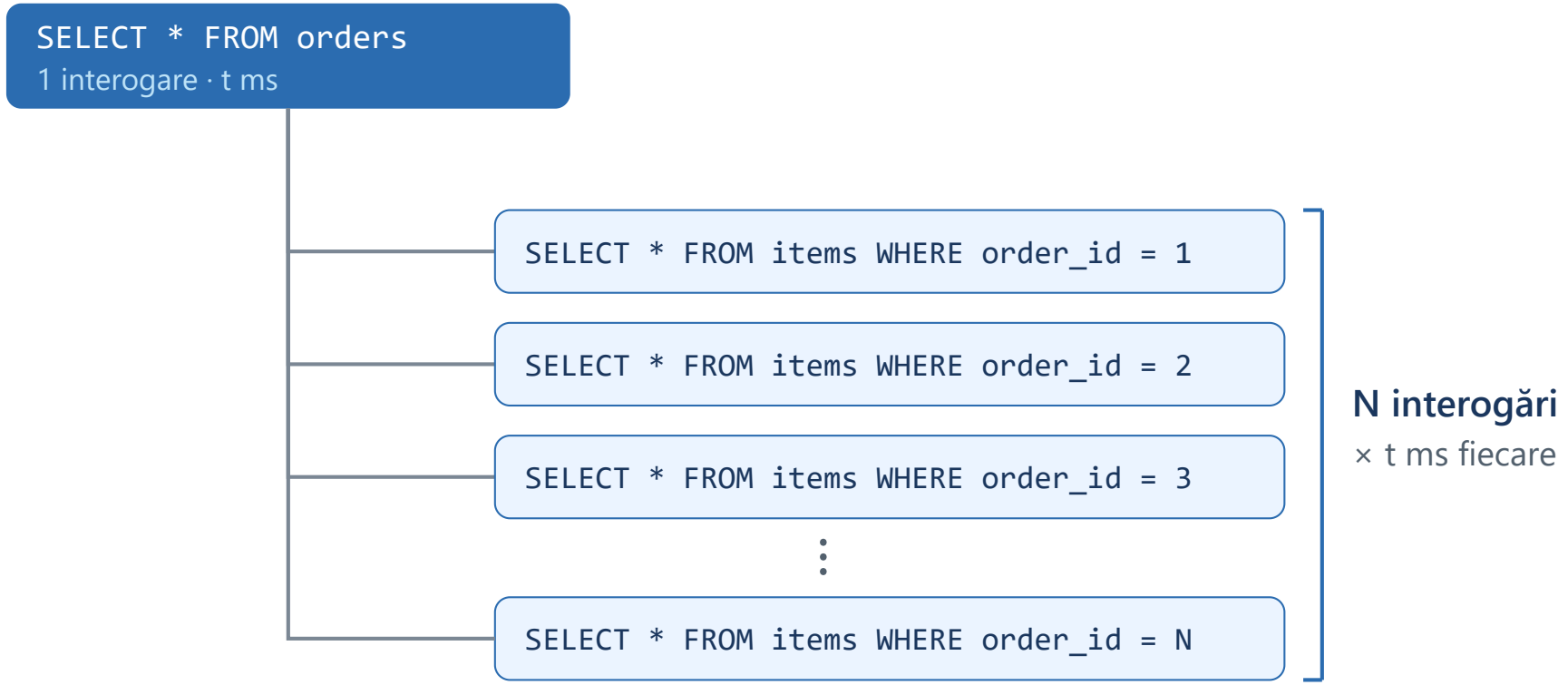
Exemplu: problema N+1 (N+1 query)

- din experiența mea, la o problemă de performanță primul lucru de verificat: avem problema N+1?

Sursa desenului: https://dev.to/dilip_v_p/the-n1-query-problem-how-one-page-fires-2101-queries-and-how-to-get-back-to-3-3l3i



O interogare costă t ms, dar generează alte N interogări



Cost total = (1 + N) × t

ex.: t = 5 ms, N = 100 → 505 ms (vs. 1 interogare cu JOIN, ~ câteva ms)

N+1: o problemă mai simplă în locul celei inițiale (problemă auxiliară)

- există biblioteci care detectează această problemă, dar presupunem că nu le folosim
- avem acces la jurnalul de SQL-uri
  - foarte multe interogări, cu atât mai multe cu cât avem mai mulți utilizatori

Cum arată în jurnalul de SQL

```
10:15:01.101  SELECT id, user_id FROM orders WHERE user_id = ?

10:15:01.104  SELECT * FROM order_items WHERE order_id = ?

10:15:01.106  SELECT name, price FROM products WHERE id = ?

10:15:01.107  SELECT name, price FROM products WHERE id = ?

10:15:01.109  SELECT * FROM order_items WHERE order_id = ?

10:15:01.110  SELECT name, price FROM products WHERE id = ?

10:15:01.112  SELECT label FROM categories WHERE id = ?

10:15:01.113  SELECT * FROM order_items WHERE order_id = ?

10:15:01.115  SELECT name, price FROM products WHERE id = ?

10:15:01.116  SELECT label FROM categories WHERE id = ?

10:15:01.118  SELECT * FROM order_items WHERE order_id = ?
```

N+1: o problemă mai simplă în locul celei inițiale (problemă auxiliară)

- clase de echivalență după numărul de repetări ale interogării: `[1]` excludem, `[2]`, `[3]` ... ne interesează
  - ordonăm clasele după numărul de repetări, iar când acesta crește, **răspunsul la întrebare iese singur în evidență** (avem problema N+1?)
- am construit o altă problemă, care se rezolvă simplu și se studiază ușor
  - putem delega către AI să extragă clasele, sau să scrie un fragment de cod care le extrage

Cum arată în jurnalul de SQL

```
10:15:01.101  SELECT id, user_id FROM orders WHERE user_id = ?
10:15:01.104  SELECT * FROM order_items WHERE order_id = ?
10:15:01.106  SELECT name, price FROM products WHERE id = ?
10:15:01.107  SELECT name, price FROM products WHERE id = ?
10:15:01.109  SELECT * FROM order_items WHERE order_id = ?
10:15:01.110  SELECT name, price FROM products WHERE id = ?
10:15:01.112  SELECT label FROM categories WHERE id = ?
10:15:01.113  SELECT * FROM order_items WHERE order_id = ?
10:15:01.115  SELECT name, price FROM products WHERE id = ?
10:15:01.116  SELECT label FROM categories WHERE id = ?
10:15:01.118  SELECT * FROM order_items WHERE order_id = ?
```

N+1: abstractizare și problemă auxiliară, nu delegare

- nu am delegat problema către AI
- am folosit abstractizarea: rândurile identice formează o clasă
- clase de echivalență după numărul de repetări ale interogării
- problema auxiliară: în loc să citim tot jurnalul, numărăm repetările pe clase
  - **răspunsul iese singur în evidență**: clasele cu multe repetări indică problema N+1

Ce diferență fac proprietățile fenomenului

- 2 exemple (deadlock, N+1): am folosit proprietățile fenomenului, nu am lăsat totul pe AI
  - am rezolvat fără AI: la deadlock fenomenul a **convergit spre starea dorită**, la N+1 **răspunsul a ieșit singur în evidență**
- dacă lăsam totul pe AI, rezultatul ar fi fost același: problema rezolvată
- diferența: persoana a învățat ceva nou și a câștigat experiență
- ce a învățat se aplică și la fenomenele viitoare (problemele, situațiile)

De la A la B: de la euristici la AI

- euristicile nu sunt exhaustive, iar unele probleme nu se rezolvă prin ele: ce rămâne, delegăm
- dar euristicile ne ajută și aici: micșorează ce delegăm
- specializarea dă un caz care se reproduce, problema auxiliară dă o sarcină îngustă, bine definită
- AI primește ceva mic și verificabil (un fragment de cod), nu problema întreagă
- exemplu, cererea fără răspuns: problema auxiliară (clase de echivalență) + AI scrie codul

Exemplu: cerere din browser fără răspuns

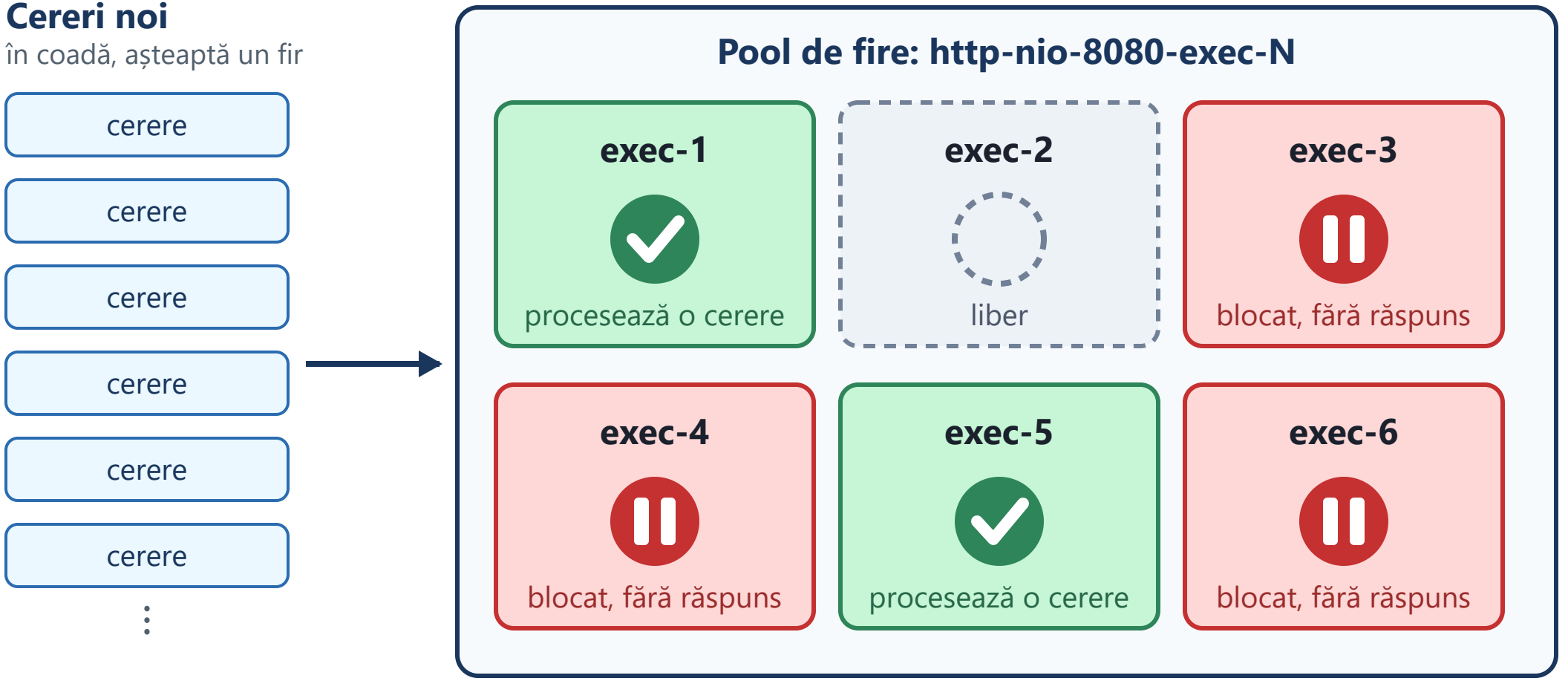
- în anumite condiții cererea din browser rămâne fără răspuns, apare aleator
- timeout-uri: presupunem că nu le avem și nu le putem seta
- avem log-ul aplicației în CSV, dar nu știm când s-a reprodus eroarea

Cerere fără răspuns: fragment de log (CSV)

```
thread,timestamp,message
http-nio-8080-exec-3,10:15:01.112,GET /orders start
http-nio-8080-exec-7,10:15:01.140,GET /report start
http-nio-8080-exec-3,10:15:01.230,GET /orders done
http-nio-8080-exec-5,10:15:02.004,GET /orders start
http-nio-8080-exec-5,10:15:02.118,GET /orders done
...
http-nio-8080-exec-3,10:16:27.377,GET /orders start
http-nio-8080-exec-3,10:16:27.490,GET /orders done
...
http-nio-8080-exec-5,10:19:41.021,GET /orders start
http-nio-8080-exec-5,10:19:41.133,GET /orders done
```

Date ilustrative. Prefixul `http-nio-8080-exec-N` = firul de execuție al serverului web (Apache Tomcat).

Cereri noi
în coadă, așteaptă un fir



Un fir blocat nu se eliberează: rămân tot mai puține fire libere, cererile noi așteaptă.

Cerere fără răspuns: aceeași metodă, clase de echivalență

- clase de echivalență după repetarea `http-nio-8080-exec-N`
- firele care n-au răspuns sunt blocate și nu pot fi reutilizate: repetările lor sunt mai puține
- datele sunt în format CSV, iar pentru el există un instrument potrivit: pandas (bibliotecă Python pentru analiza datelor tabelare)
- îi cer AI-ului codul pandas, nu analiza în sine
- adaptez codul (dacă e cazul) și obțin un instrument determinist

Cerere fără răspuns: promptul pentru codul pandas

> Am o listă de înregistrări într-un CSV, cu coloane thread (firul de execuție), timestamp și message (mesajul). Dă-mi un cod pandas care afișează fiecare thread cu ultimul timestamp văzut într-un anumit interval de timp.

Cerere fără răspuns: codul pandas

```
import pandas as pd

df = pd.read_csv("log.csv", parse_dates=["timestamp"])

start, end = "10:15:00", "10:20:00"
interval = df[(df["timestamp"] >= start) & (df["timestamp"] <= end)]

last_seen = interval.groupby("thread")["timestamp"].max()
print(last_seen.sort_values())
```

Firele cu ultimul timestamp cel mai vechi, primele în listă, sunt candidații pentru „blocate”.

Cerere fără răspuns: ce afișează codul

```
thread
http-nio-8080-exec-7   2026-11-13 10:15:01.140
http-nio-8080-exec-3   2026-11-13 10:16:27.490
http-nio-8080-exec-5   2026-11-13 10:19:41.133
Name: timestamp, dtype: datetime64[us]
```

Firul `exec-7` e primul: ultima activitate a lui e la 10:15:01, cu aproape 5 minute înaintea celorlalte. El e firul blocat.

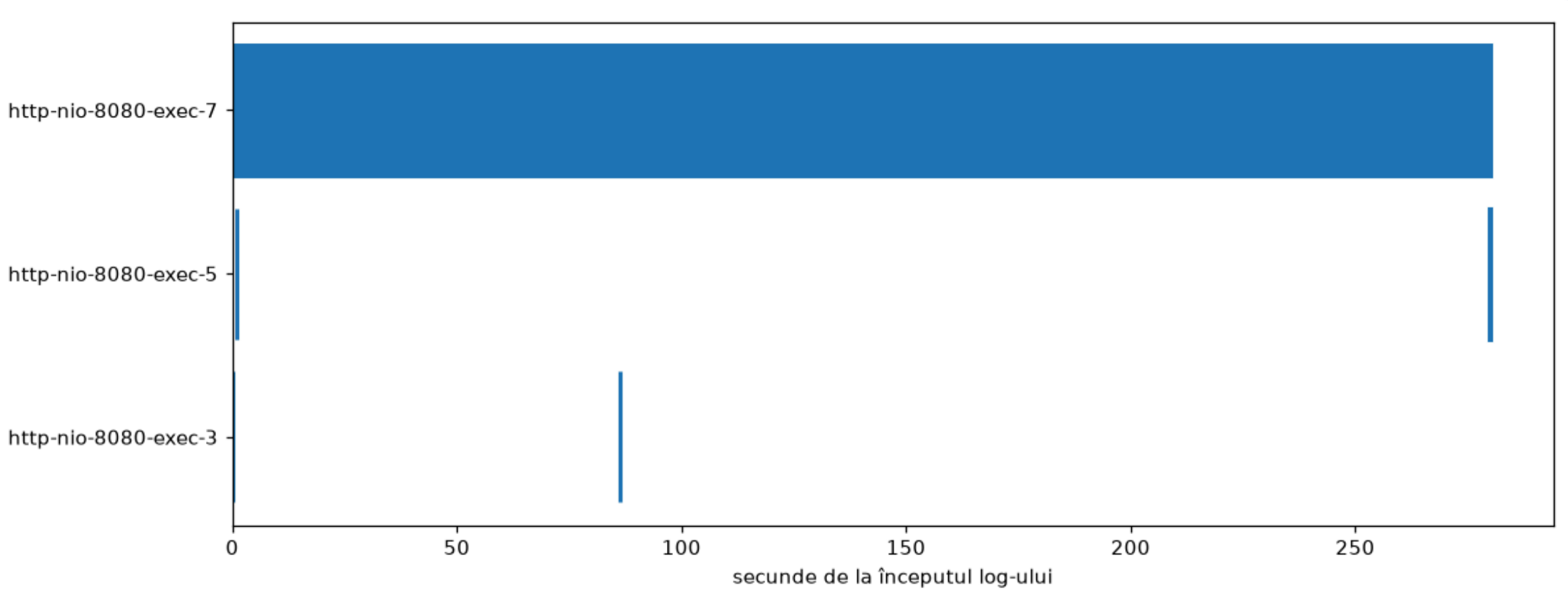
Cerere fără răspuns: a doua soluție, desenăm o figură

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("log.csv", parse_dates=["timestamp"])
df["op"] = df["message"].str.split().str[-1]  # start / done
df["req"] = df.groupby(["thread", "op"]).cumcount()

w = df.pivot(index=["thread", "req"], columns="op", values="timestamp").reset_index()
begin = (w["start"] - df["timestamp"].min()).dt.total_seconds()
length = (w["done"].fillna(df["timestamp"].max()) - w["start"]).dt.total_seconds()

plt.barh(w["thread"], length, left=begin, edgecolor="C0", linewidth=2)
plt.xlabel("secunde de la începutul log-ului")
plt.show()
```



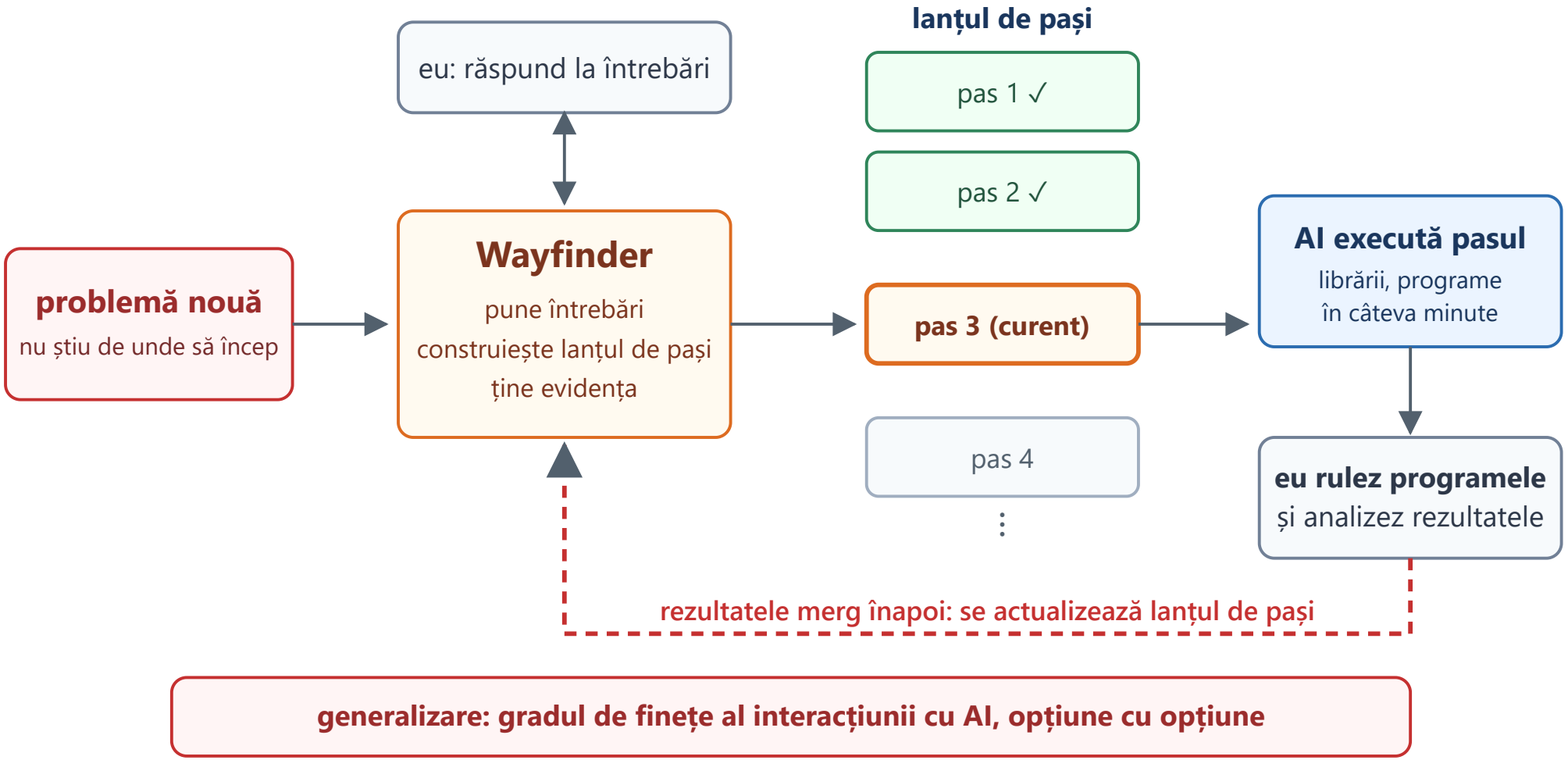
Fiecare bară = o cerere, de la start la done. Cererile terminate sunt linii scurte; doar `exec-7` are o bară care nu se mai termină.

B. Exemplu: skillul /wayfinder pentru situații necunoscute

- problemă nouă: nu știu de unde să încep (formulare neclară sau primii pași costisitori)
- Wayfinder pune întrebări, construiește lanțul de pași, ține evidența
- AI execută pașii tehnici (librării, programe) în câteva minute
- eu rulez programele și analizez rezultatele
- generalizare: **gradul de finețe în interacțiunea cu AI**
  - înlocuiesc prezentarea tuturor opțiunilor cu prezentarea opțiune cu opțiune

Sursa: https://github.com/mattpocock/skills/tree/main/skills/engineering/wayfinder

Wayfinder: de la o problemă neclară la pași pe care îi execut



Situații necunoscute: AI e foarte bună la enumerare

Problema nouă, pasul următor: ce opțiuni am?

- întrebări la care nu m-am gândit
- riscuri și cazuri limită
- alternative și variante
- ipoteze ascunse
- criterii de comparare
- ce aș putea să fi uitat
- exemple și contraexemple

AI enumeră și ce nici nu știam că există, iar astfel **ne orientăm spre direcția corectă** de rezolvare a problemei.

Exemplu: schimbăm modul de prezentare, cerem și explicația vizuală de la AI

Problema nouă, pasul următor: înțeleg rezultatul repede.

- articol: „How to 5x your communication effectiveness with Claude Code” (Towards Data Science)
- agentul scrie rezultatele netriviale ca fișier HTML, deschis în browser, nu ca text în terminal
- instrucțiunea se pune o dată, în fișierul MD la nivel de utilizator
- skill-uri pentru sarcini concrete (ex. raport de teste)
- concluziile apar la final, rezultatul e prezentat vizual

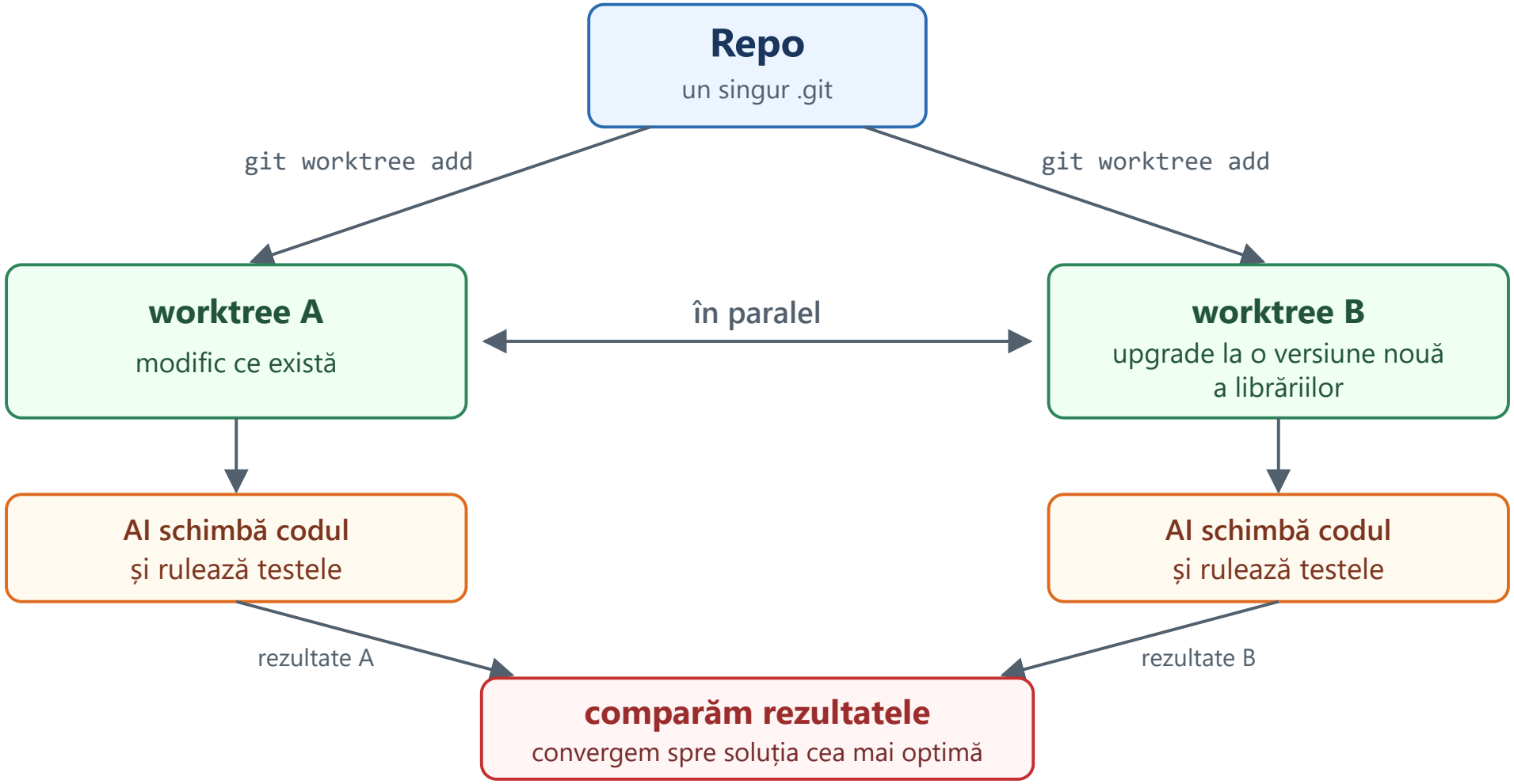
Explicația vizuală: ce iese în evidență

- explicația vizuală ne ajută să **înțelegem rezultatul mai repede**
- ce contează **iese singur în evidență**, în loc să fie ascuns în text lung
- astfel ne orientăm mai repede spre direcția corectă

Exemplu: analiza unei probleme de performanță – git worktree

- problemă de performanță
- opțiuni: modific ce există sau fac upgrade la o versiune nouă a librăriilor
- ce aleg? cum măsor?
- git worktree: experimentăm, salvăm rezultatele, comparăm

git worktree: două experimente în paralel, din același repo

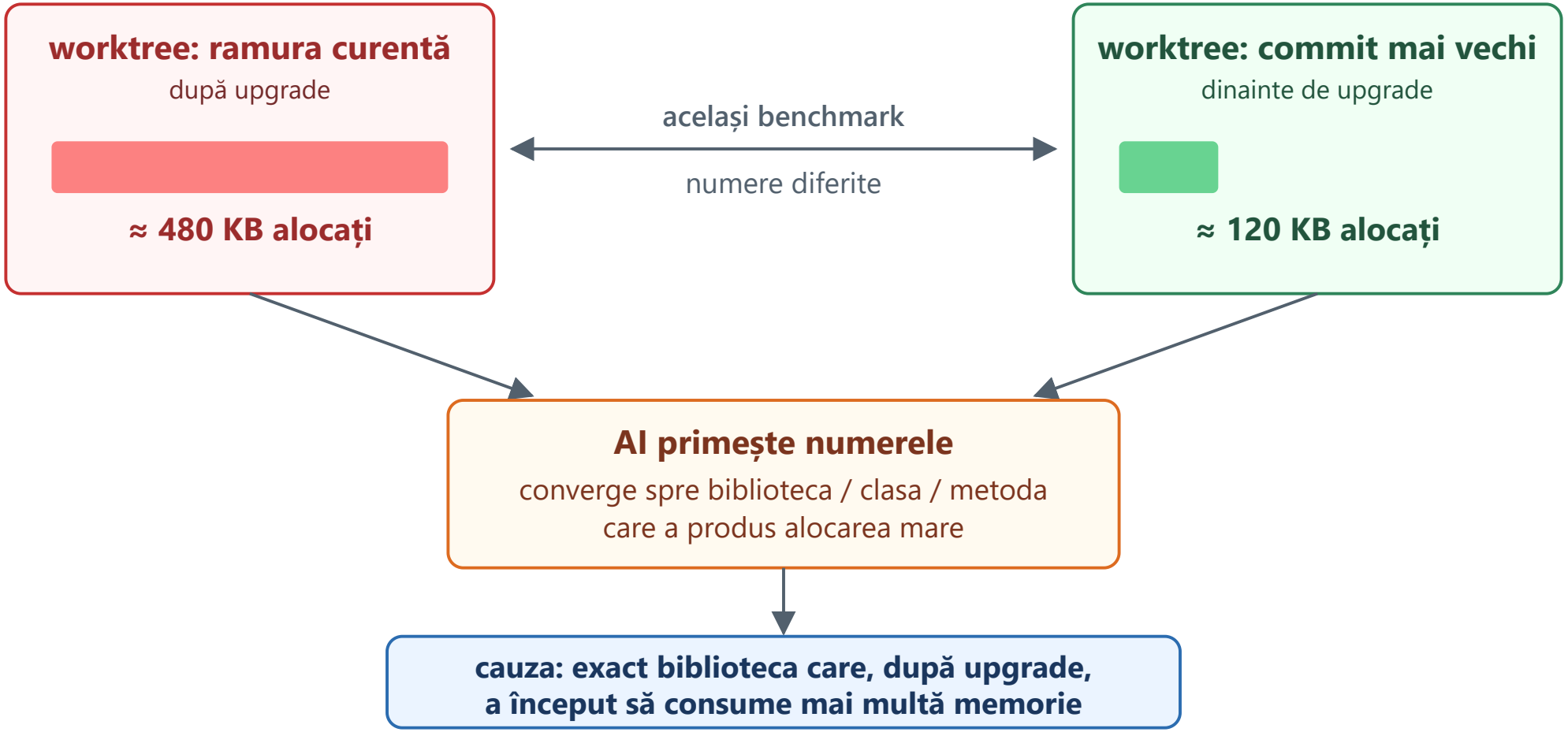


Exemplu: modificarea consumului de memorie după upgrade

- după upgrade, aplicația consumă mai multă memorie
- **AI generează un benchmark** care, pentru un endpoint de API, măsoară și salvează numărul de bytes alocați
- **git worktree**: același benchmark rulează pe ramura curentă și pe un commit dinainte de upgrade
- numerele diferă, deci avem o referință măsurabilă
- **AI primește numerele** și caută cauza
  - a găsit exact biblioteca care, după upgrade, a început să consume mai multă memorie

git worktree: același benchmark pe două versiuni

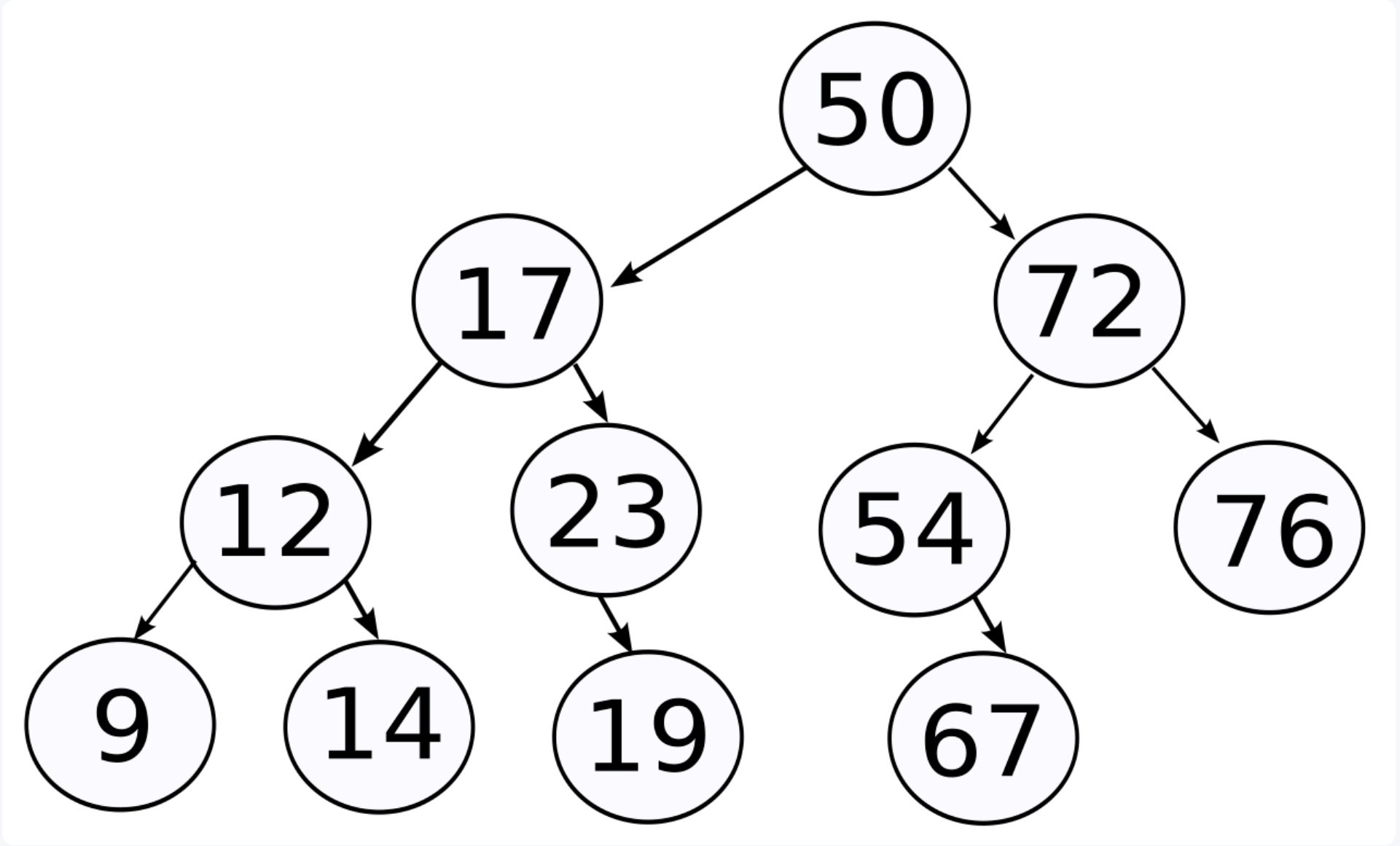
Date ilustrative: memoria alocată de un endpoint de API, pe o cerere.



Exemplu: API care întoarce un arbore prea mare

- API-ul întoarce o structură arborescentă, prea mare: programul pică
- soluția propusă de AI (răspuns mediu): introduceți paginarea
- se poate, dar răspunsul nu mă aranjează și nu mă grăbesc să-l aplic
    - paginarea pe arbore e mai complexă decât pe o structură liniară

Sursa desenului: https://commons.wikimedia.org/wiki/File:Tree-data-structure.svg



Arbore prea mare: delegăm AI generarea soluțiilor posibile și verificarea

- **construiesc teste** care folosesc API-ul și pică
- **AI marchează toate locurile** unde e folosit API-ul, făcându-l să returneze eroare
- **studiez fiecare loc** în parte
  - uneori există deja un API de căutare
  - alteori sunt de ajuns copiii de primul nivel, nu tot arborele
- **AI încearcă fiecare soluție**: schimbă codul și rulează testele
  - eu verific dacă testele pică în aceleași condiții

Arbore prea mare: enumerarea variantelor, AI umple lista

Structura arborescentă e prea mare. Variante:

- paginare
- limitarea adâncimii (doar copiii de primul nivel)
- încărcare lenește (lazy loading) a subarborilor
- structură plată (listă de noduri cu `parentId`)
- selectarea câmpurilor (doar ce e necesar)
- API de căutare / filtrare în loc de întregul arbore
- streaming în loc de un singur răspuns

Exemplu: dashboard-uri Grafana generate de AI

- **nevoia**: monitorizăm ceva în Grafana, cu câteva dashboard-uri noi
- **formatul**: dashboard-urile Grafana sunt fișiere JSON
- **AI generează** JSON-ul, iar eu îl import în Grafana
- **rezultatul**: sunt scutit de munca manuală de a crea dashboard-urile

Sursa desenului: https://commons.wikimedia.org/wiki/File:Grafana_screenshot_(2018).png



Ce se poate reprezenta digital, AI produce rapid

- în mediul digital, tot ce se poate reprezenta pur digital (un widget JSON, un dashboard, un fișier de configurare) AI produce în câteva secunde
- același rezultat, manual, înseamnă o secvență de pași: interfață, click-uri, configurări, verificări
- rezultatul e un artefact pe care îl import și îl verific eu

Exemplu: autentificare automată în teste

- **problema**: testele necesită autentificare
- **script Playwright de login**, construit de AI, obține un bearer token
- **credențialele** sunt într-un fișier `.env`, iar tokenul se salvează în `auth.json`
- testele citesc automat `auth.json`
- dacă tokenul a expirat, se pornește singur scriptul Playwright și obține unul nou
- **rezultatul**: totul merge determinist, fără intervenția mea

Teste cu autentificare automată



nu mai e nevoie de intervenția mea
totul merge determinist

Exemplu: delegăm AI telemetria

- rulăm testele (sau chiar procesul) și colectăm:
  - **log-urile** aplicației
  - **metrici Prometheus**
  - **flamegraph** (unde își petrece programul timpul)
- delegăm toate acestea către AI
- o singură întrebare: **„Vezi ceva anormal în aceste date?”**
- AI ne ajută să **convergem foarte repede** spre cauza problemei

De ce nu delegăm totul către AI?

- problemele simple, de rutină, le delegăm deja
- AI se descurcă bine și cu cele complexe
- deci de ce nu delegăm întreaga complexitate?
  - dăm problema, luăm răspunsul
- **răspunsul din exemplele de mai sus**: abordarea mea e să stabilesc corect ce parte delegăm AI și ce parte nu

Ce pierdem dacă delegăm totul: modelul mental

- înțelegerea unui fenomen cere un model mental al acestuia
- sarcini simple: modelul mental există deja
  - nu pierdem nimic delegând către AI
- sarcini complexe: modelul mental încă nu există
  - delegându-le către AI, pierdem șansa să ne formăm modelul
- formarea modelului mental necesită timp
- noi decidem: ne trebuie modelul sau nu? (raportul timp:rezultat cu AI e foarte bun)

De ce îți trebuie modelul mental?

- **abordare reutilizabilă**: un fenomen complex înțeles bine ne ajută să abordăm mai ușor alte fenomene din aceeași familie
- **calitatea rezultatului**: raportul timp:rezultat nu e suficient
  - rezultatul poate fi **neoptim** (multe iterații): trebuie să schimbăm felul în care AI abordează problema
  - rezultatul poate fi **incorect**, pentru că AI e non-deterministă: avem nevoie de subrezultate deterministe
- ambele cer mai mult decât un model superficial al fenomenului

> **Interviewer:** “I heard you were extremely quick at math.”
>
> **Me:** “Yes, as a matter of fact I am.”
>
> **Interviewer:** “What is 14 × 27?”
>
> **Me:** “49.”
>
> **Interviewer:** “That’s not even close.”
>
> **Me:** “Yeah, but it was fast.”

Glumă populară pe internet.

Experiment: „copiază imaginea” de 101 ori

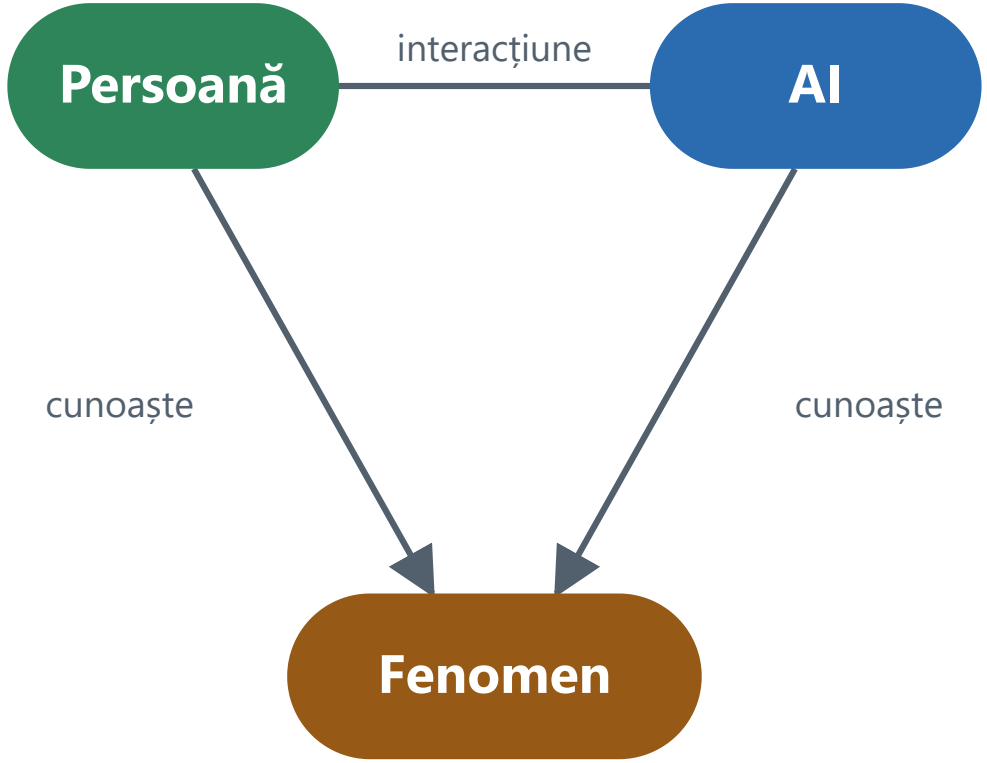
- postare pe Reddit (r/ChatGPT): aceeași poză cu Dwayne Johnson, recreată de ChatGPT de 101 ori, de fiecare dată pornind de la rezultatul anterior
- cerința: „Create the exact replica of this image, don't change a thing”
- primele variante seamănă cu originalul, apoi trăsăturile se deformează până la artă abstractă

Postarea originală (Reddit, u/Foreign_Builder_2238) · publicații similare: TechRadar, Know Your Meme, Daily Dot, Kapwing

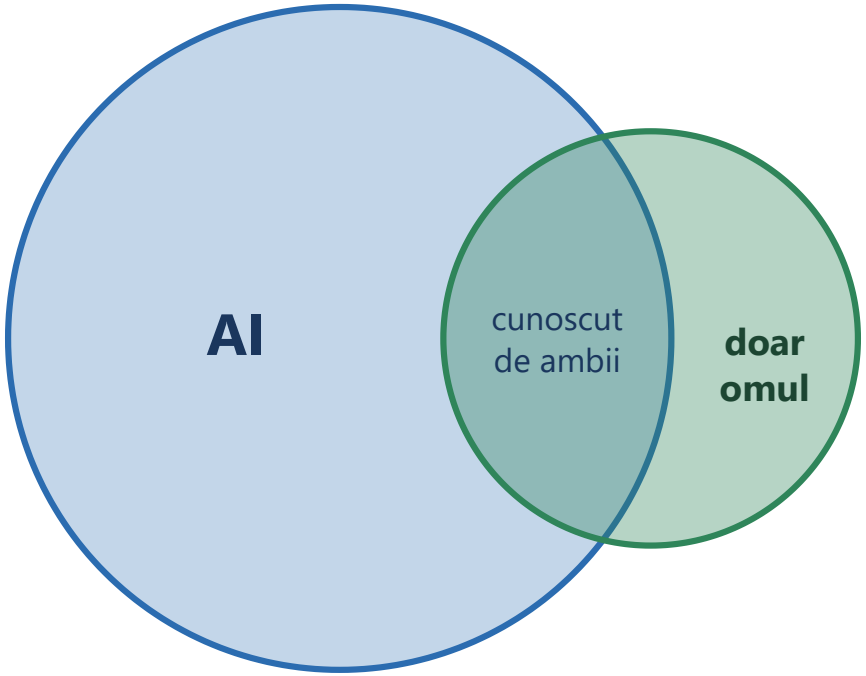
Al treilea participant: modelul (fenomenul)

- **polarizarea de azi**: de o parte omul, de cealaltă AI
- **al treilea participant**: modelul (fenomenul)
  - poate fi un model mental, un model software, un cadru științific, cunoștințe, ...
- **la îndemână**: le putem folosi direct, fără AI

Persoană, AI și fenomen



Cunoștințe despre fenomen



sfera AI e mare, sfera omului mai mică,
dar nu e cuprinsă toată în AI

Regula mea

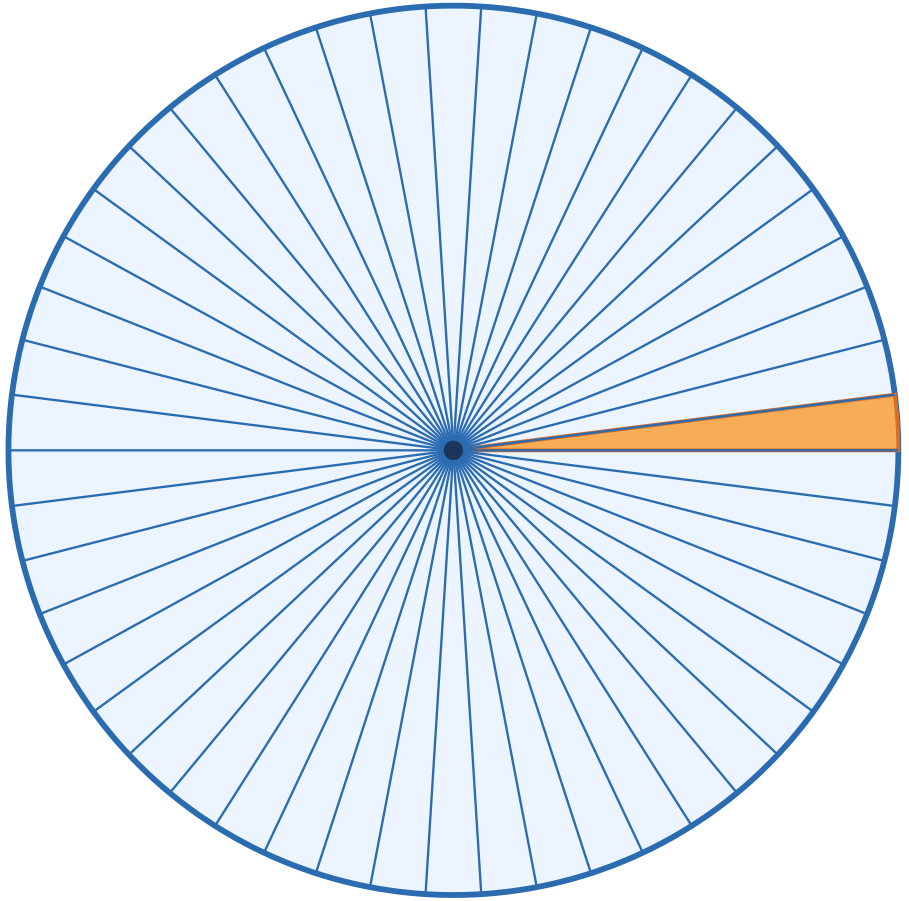
- **primul pas**: poate modelul fenomenului să rezolve problema cu efort minim?
- **dacă pot amândoi** (modelul fenomenului și AI), nu folosim AI
  - proprietățile fenomenului fac ca acesta să **convergă singur spre starea dorită**, iar soluția iese în evidență
- AI poate rezolva multe probleme, dar **nu e singurul participant** care poate: poate și modelul

Imaginile din această prezentare fără sursă indicată (inclusiv cea de pe slide-ul de titlu) au fost generate cu ajutorul Claude Code.

În loc de concluzie: cum a măsurat Eratostene Pământul cu un simplu băț

- **Eratostene din Cirene** (cca 276–194 î.Hr.), în jurul anului 240 î.Hr.
- la solstițiul de vară, la amiază, în Syene (azi Assuan, în Egipt) soarele era la zenit: bățul nu avea umbră
- în același moment, în Alexandria, bățul făcea cu razele Soarelui un unghi de ≈ 7,2° (măsurat după umbră), adică 1/50 dintr-un cerc
  - un cerc întreg are 360°, iar 360° : 7,2° = 50
- razele Soarelui sunt **paralele**, deci același unghi apare și la centrul Pământului, între cele două orașe

De ce 7,2° înseamnă 1/50 dintr-un cerc



cercul împărțit în 50 de felii egale

cercul întreg: **360°**

o felie: **7,2°**

360° : 7,2° = 50

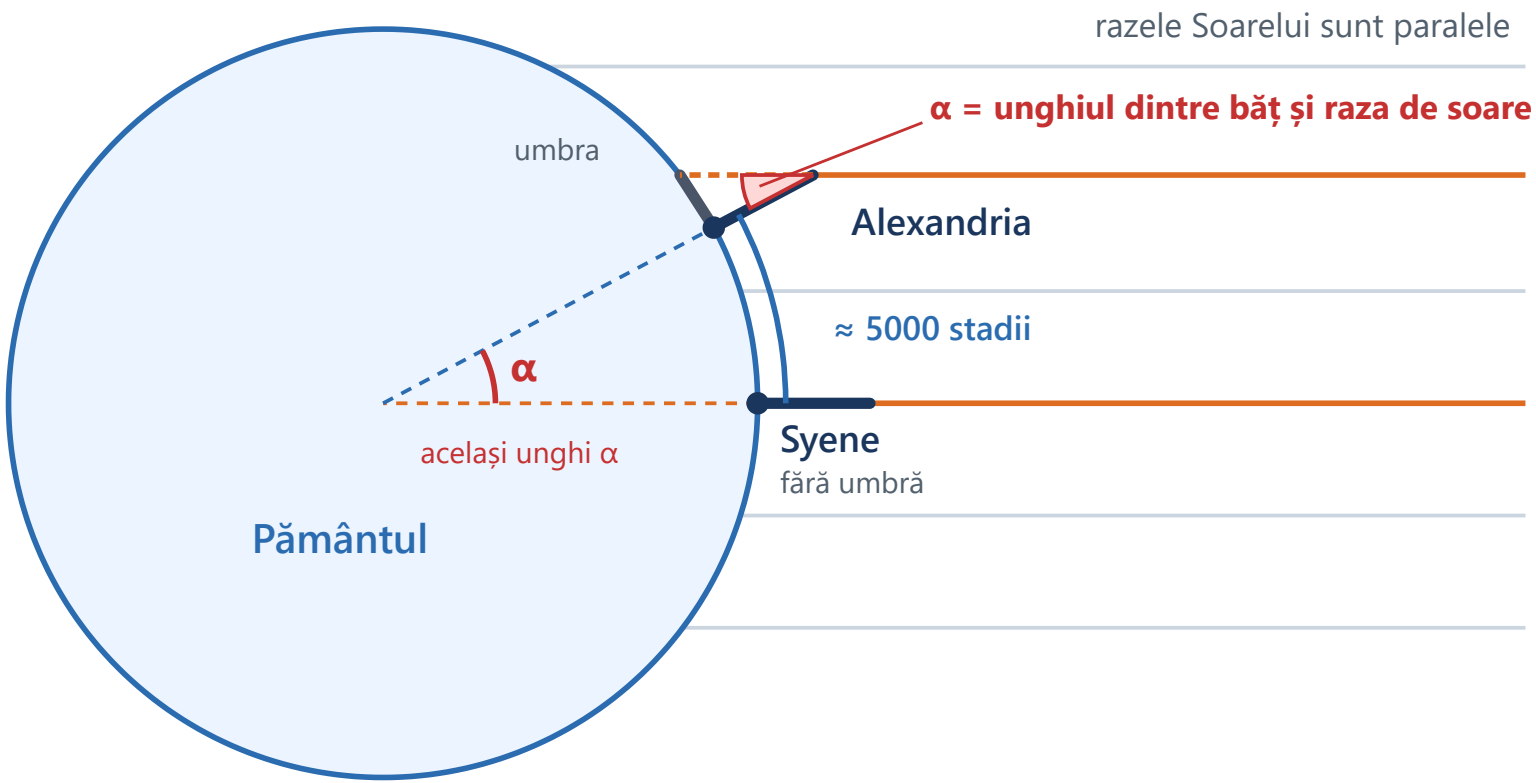
deci 7,2° = **1/50** dintr-un cerc

distanța dintre Syene și Alexandria este o singură felie

În loc de concluzie: Eratostene, stadii și circumferința

- **stadiu** = unitate de lungime din Grecia antică, după lungimea pistei de alergare dintr-un stadion
  - stadiul egiptean ≈ 157,5 m, cel atic ≈ 185 m: nu se știe sigur care a fost folosit
- distanța dintre orașe: ≈ 5000 de stadii (≈ 800–925 km)
- circumferința Pământului: ≈ 50 × 5000 = **250 000 de stadii** (≈ 39 000–46 000 km; valoarea reală: ≈ 40 075 km)

Eratostene: Pământul măsurat cu un băț și o umbră



α ≈ 7,2° = 1/50 dintr-un cerc, deci circumferința ≈ 50 × 5000 = 250 000 stadii

unghiul dintre băț și raza de soare (în Alexandria) este egal cu unghiul dintre cele două orașe, văzut din centrul Pământului

Câți tokeni a cheltuit Eratostene din Cirene?